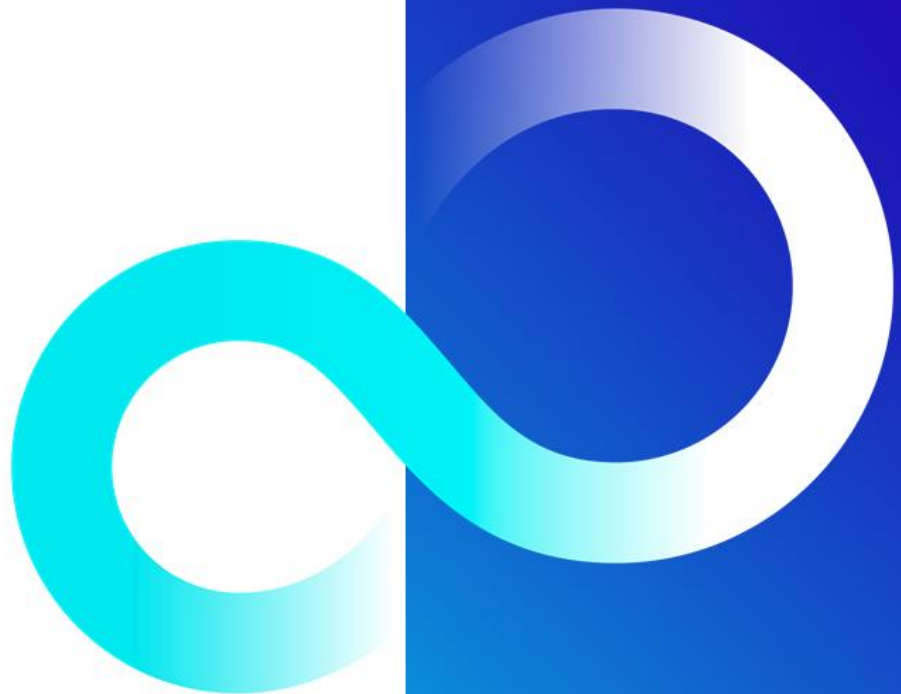


Update Set Mover Application

FUJITSU



Development Document

Developed and created by Pedro Leite

Select Information Classification © Fujitsu 2022

Contents

1. Overview	5
1.1 Purpose of Document	5
1.2 Resume of the Application	5
2. Important Notes.....	6
3. Global Scope.....	7
3.1 Script Include.....	7
3.1.1 UpdateSetAjaxGlobal.....	7
3.2 Choices	7
3.2.1 Pending Completion on this instance	7
3.3 Dictionary.....	7
3.3.1 Completed on Prod	7
3.3.2 Ready to move to prod	7
3.4 Client Script.....	7
3.4.1 Show options if US is committed on Prod.....	7
3.4.2 Update Set Mover State Options	8
3.5 UI Action.....	8
3.5.1 Mark Update Sets As Ready to Go to Prod.....	8
4. Application Scope	9
4.1 Modules	9
4.1.1 Application Properties	9
4.1.2 Instances Configurations	9
4.1.3 Ready to Move to Prod.....	9
4.1.4 Update Set State Histories.....	9
4.1.5 USM – Business Rules.....	9
4.1.6 USM – Script Include	9
4.1.7 USM – Client Script.....	9
4.2 System Activation.....	9
4.3 Tables	10
4.3.1 Instances.....	10
4.3.2 Update Set State History	11
4.4 Business Rules.....	11
4.4.1 If committed on Prod update last instanc.....	11

4.4.2 Turn On / Off the Whole Application	11
4.4.3 Commit Update Set Log to History.....	12
4.4.4 Set States after Update Set is Committed.....	12
4.5 Script Include.....	12
4.5.1 updateSetMoverUtils.....	12
4.6 Roles	12
4.6.1 Instance Configuration table.....	12
4.6.2 12	
4.6.3 Out of Box Admin Role	12
4.6.4 instances_user.....	12
4.6.5 update_set_state_history_user.....	12
4.6.6 update_set_mover_admin	13
4.7 System Properties	13
4.7.1 userID	13
4.7.2 password	13
4.7.3 get_instance_name	13
4.7.4 Application Switch On/Off	13
4.8 Inbound Integrations	13
4.8.1 Scripted REST APIs.....	13
4.8.1.1 Remove Flag from Committed Update Sets.....	13
4.8.2 Scripted REST Resource.....	13

1. Overview

1.1 Purpose of Document

The purpose of the document is to highlight the development work of the Update Set Mover Application.

This document has an explanation about each record that makes the whole application work, but it doesn't show any scripts whatsoever, only the purpose, in a resume, of the scripts, purpose, logic and functionalities.

1.2 Resume of the Application

It's a simple application that:

- Allows an extra layer of approval for the Update Set to go to Production and
- Records all the operations on a table called "Update Set Mover History"

This will allow to:

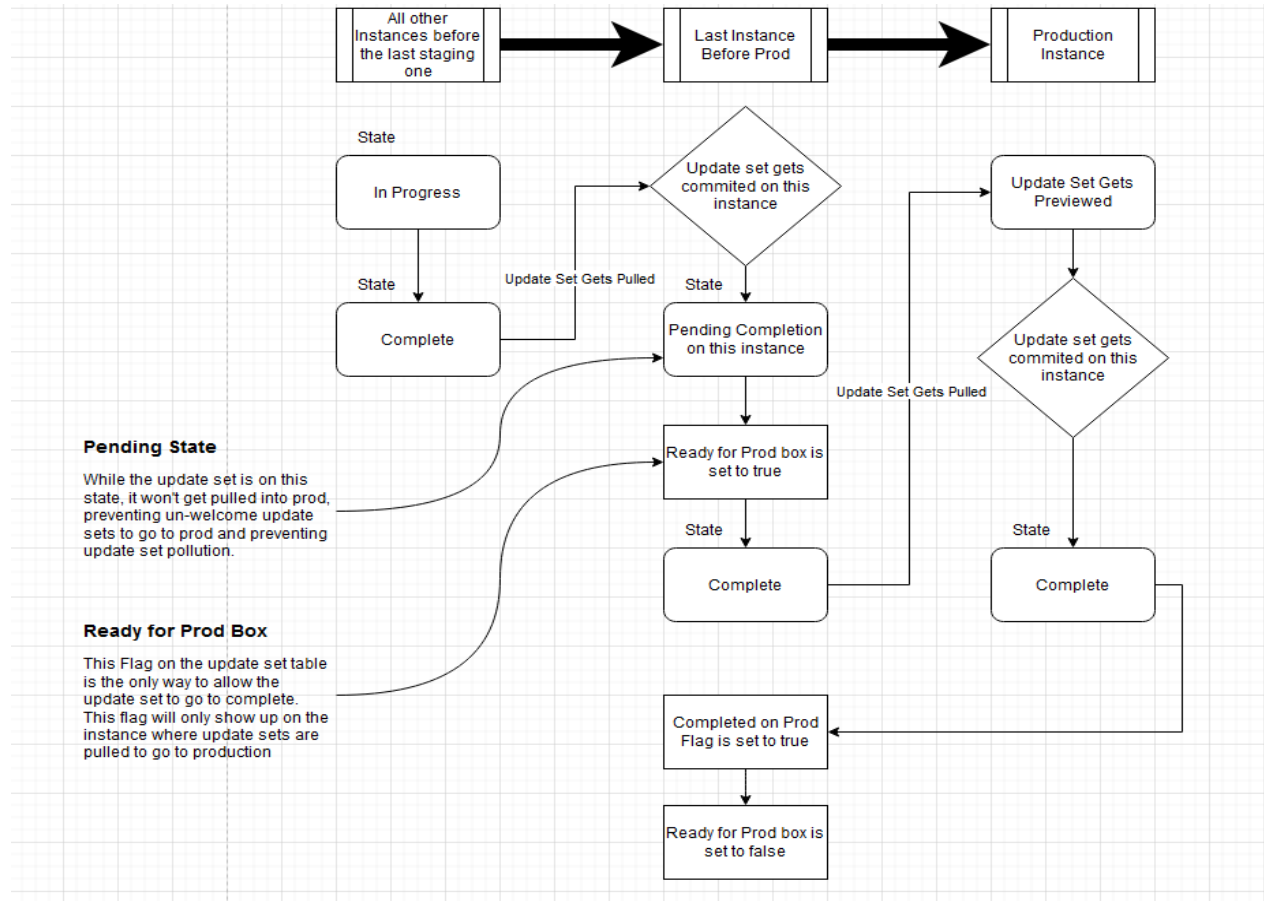
- Reduce "update set" pollution on production
- More control on what goes to production and have a better way to track who created, pulled, committed, or backed out the updates sets on all instances where this is installed

It is also possible to create reports based on the "Update Set Mover History"

2. Important Notes

For this application to work, there must be a remote instance configuration already in place, at least two instance, Development and Production instance.

The following graphic illustrates how everything works in theory.



This application is divided into two pieces, the global update set and the application scope update set. The Global update set contains a Script include that it will be the bridge between Global and Application Scope. This was split due to access restriction to the sys_update_set table outside of the global scope.

Always make sure you're in the correct scope while doing stuff on the application, for example, if you need to turn on/off the whole application in the application properties, you must be on the application scope.

3. Global Scope

These records are generated from a specific script on global scope, but it's called from the application scope.

3.1 Script Include

3.1.1 UpdateSetAjaxGlobal

Various ajax functions for update sets from Update Set Mover application.

It contains various methods such as:

- **setUpdateSetStateAfterCommitted** sets the update set to "Pending Completion on this instance" after the update set is committed.
- **buildRecords** builds records after the application is turned on.
- **deleteRecords** deletes the records that were created after the application is turned on.
- **setReadyForProdFalseGlobal** Sets the flag "ready for prod?" to false after being committed on prod on the current instance (if this is the one before production and configured on the Instances table).
- **checkIfLastInstanceBeforeProd** checks if the current instance is the last one before production.

It also has some methods that helps the connection between instances and allows the flag "Completed on prod" and "Ready to move to prod?" to be disable or enable automatically.

The methods to build and delete records, generate the records on 3.2, 3.3, 3.4 and 3.5.

3.2 Choices

3.2.1 Pending Completion on this instance

Label: "Pending Completion on this instance"

value: "pending"

3.3 Dictionary

3.3.1 Completed on Prod

Label: "Completed on Prod"

value: "u_completed_on_prod"

3.3.2 Ready to move to prod

Label: "Ready to move to prod?"

value: "u_ready_to_move_to_prod"

3.4 Client Script

3.4.1 Show options if US is committed on Prod

On Load and On Change Client script based on "Ready to move to prod?" field.

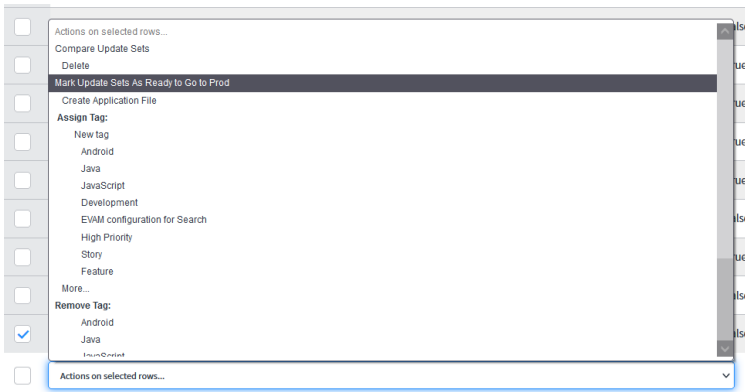
3.4.2 Update Set Mover State Options

Update Set Mover State Options depending on the instance options configurations from the Update Set Mover application.

3.5 UI Action

3.5.1 Mark Update Sets As Ready to Go to Prod

List view UI action that allows a bulk change of state to complete and set the flag "Ready to move to prod?" to true. This allows multiple Update Sets to be set simultaneously.



4. Application Scope

The application scope is where most of the application was built.

4.1 Modules

4.1.1 Application Properties

Core configurations to setup the Application, this has to be configured first before the application is used.

4.1.2 Instances Configurations

This is where we configure how the application will behave in each instance, it will be explained on [4.2](#).

4.1.3 Ready to Move to Prod

Shortcut to all update sets that have the flag "Ready to Move to Prod" set to true. This might help the developers or whoever needs to raise Changes for deployment.

4.1.4 Update Set State Histories

Update set state history table is where all the state updates of the update sets are stored.

4.1.5 USM – Business Rules

Shortcut to the Business Rules used by the application.

4.1.6 USM – Script Include

Shortcut to the Script Include used by the application.

4.1.7 USM – Client Script

Shortcut to the Client Scripts used by the application.

Update Set Mover	★
Application Properties	★
Instances Configurations	★
Ready to Move to Prod	★
Update Set State Histories	★
USM - Business Rules	★
USM - Script Include	★
USM - Client Script	★

4.2 System Activation

This is a configuration record that has some core configurations of the application such as:

- User ID to make the REST call to update the state in the previous instance. This user ID must exist on the last instance before prod, so the update set can be updated via REST call.
- password to make the REST call to update the state in the previous instance
- This property allows you to turn ON or OFF the entire application. Write true or false.

Update Set Mover

The following property sets the application on/off

This property should only be switched off if the application isn't being used.

Make sure you're on the Update Set Mover Application before you switch off/on the application.

User ID to make the REST call to update the state in the previous instance. This user ID must exist on the last instance before prod, so the update set can be updated via REST call. ?

password to make the REST call to update the state in the previous instance ?

This property allows you to turn ON or OFF the entire application. Write true or false. ?

Save

4.3 Tables

4.3.1 Instances

The table Instances (x_661539_update_0_instances) is where the Instances configurations are stored.

Each record stores the proper information for the whole application to work.

Fields and explanation:

Name: name of the instance

Order of instance: The lowest number should always be production, the next one should be the instance where update sets get pulled and the others should be the Development or other instances.

So, for example, on a client that uses Prod, Dev and Test, and where Test is the last instance before prod, the instances should be configured the following way -> Prod (100), Test (200) and Dev (300).

Type: Type of instance, it has 3 options, Production, Development and Test.

Update Sets Get Pulled to Prod from This instance?: Set this to true if the instance is the last one before production, which means, the update sets goes from here to production.

Record Example

Instances

true

Update

Delete

* Name

dev69957

* Order of Instance

100

* Type

Production

Active

☒

Update Sets Get Pulled to Prod from This Instance?

☐

Created

2021-12-03 09:40:17

Update

Delete

4.3.2 Update Set State History

This table records all the state changes applied to the update sets. This table is extremely important since it's the one that triggers the update of the flag "Completed on Prod" and sets the "Ready to move to prod?" flag to false.

Fields and explanation:

Number: Record number

Update Set Name: Name of the target update set

Update set SysID: Original sysid of the update set. This value will be moved, as Out-of-Box, from instance to instance and it's the value that allows the application to find the update set in all instances where it exists.

State: State

Updated From: Update source where this update set came from.

Updated by: Person that made the update to the update set.

Completed by: Person that completed the update set (in case the state was committed).

Description: Description of the update set.

Record Updated by: Person that updates the update set (in case the state wasn't committed).

Current Instance Type: The type of the current instance.

Record Example

Update Set State History

PMGL - Prod

Number

USSH0010078

Update Set Name

Test for Dev

Update Set SysID

c84d94ac2f31011058d5fe1df699b6db

State

Committed

Updated from

PMGL - Prod

Updated by

admin

Completed by

admin

Description

Record Updated By

Current Instance Type

Development

<date

Delete

4.4 Business Rules

4.4.1 If committed on Prod update last instanc

This business rule is based on the Update Set State History table and, if the current instance is Production and the state of the record is "committed", it makes a REST call to the previous instance, configured on the Instance table, search for the update set there and updates the flag "Completed on Prod" to true and sets the flag "Ready to move to prod?" to false.

4.4.2 Turn On / Off the Whole Application

Business rule responsible for listening when the flag "This property allows you to turn ON or OFF the entire application. Write true or false." In the Application properties changes to true or false.

If true, turns the application on and creates the records described on point [3.1.1](#).

4.4.3 Commit Update Set Log to History

Business rule responsible to create a record on the Update set history table every time the state of the update set changes to Preview, committed, or backed out.

4.4.4 Set States after Update Set is Committed

Business rule responsible to set the state of the update set to "Pending Completion on this instance" if the instance is the last one before production and the update set was committed on that instance.

This business rule triggers on insert when the state is complete, and the remote sys id isn't empty.

This situation only happens when an update set is retrieved, and it's committed on that instance. This will make the update set to be inserted into the sys_update_set with a "Original sys_id" and a "Remote Sys Id".

4.5 Script Include

4.5.1 updateSetMoverUtils

This script include has all the scripts that run on the server side of the application.

It contains various methods such as:

- **getCurrentInstanceValue** to get the current instance value from the Instances table.
- **switchFunction** to switch the application on/off.
- **_buildOrDeleteGlobalRecords** to tell the global script, based on if it's to turn or not the application, to build or delete records into the global scope.
- **_getPreviousInstance** to check the information's of the "previous" instance on the Instances table.
- **updatePreviousInstanceState** calls the method described in [4.8](#) that updates the flag "Ready for prod?" and "Completed on Prod" in the previous instance before production configured on the Instance table.
- **setReadyForProdFalse** Sets the flag "ready for prod?" to false after being committed on prod on the current instance (if this is the one before production and configured on the Instances table). Calls the method described in [3.1.1](#) on the global scope.
- **_writeLog** that works as a specific log tag to be easier to identify on the log table ([USM] + string)

4.6 Roles

4.6.1 Instance Configuration table

4.6.2

4.6.3 Out of Box Admin Role

Has access to everything.

4.6.4 instances_user

Has access to the Instance table with read and write access.

4.6.5 update_set_state_history_user

Has access to the Update Set State History with read permission.

4.6.6 update_set_mover_admin

Has access to the Instance table and the Update Set State History with read/write/create/delete permission.

4.7 System Properties

4.7.1 userID

User ID to make the REST call to update the state in the previous instance. This user ID must exist on the last instance before prod, so the update set can be updated via REST call.

4.7.2 password

Password to make the REST call to update the state in the previous instance

4.7.3 get_instance_name

Get current instance name

4.7.4 Application Switch On/Off

This property allows you to turn ON or OFF the entire application.

Write true or false.

4.8 Inbound Integrations

4.8.1 Scripted REST APIs

4.8.1.1 Remove Flag from Committed Update Sets

Scripted REST Service

Remove Flag from Committed Update Sets

Update

Delete

* Name

Remove Flag from Committed Update Sets

* API ID

remove_flag_from_committed_update_sets

Active

☒

Protection policy

-- None --

Application

Update Set Mover

API namespace

x_661539_update_0

Base API path

/api/x_661539_update_0/remove_flag_from_committed_upd

Security

Content Negotiation

Documentation

Default ACLs may be selected to apply to all resources, but individual resources can override this setting. The Default ACLs are enforced for a resource when:

- The resource 'Requires authentication' and 'Requires ACL authorization' fields are selected, and
- The resource itself does not reference any ACL records

Access is granted if at least one matching ACL record is found.

[More info](#)

Default ACLs

🔒

✖ Scripted REST External Default

Update

Delete

Related Links

[Enable versioning](#)
[Explore REST API](#)
[API analytics](#)

Resources (1)

Request Headers

Query Parameters

Resources

New

Search

Name

Search

API definition = Remove Flag from Committed Update Sets

⚙️	🔍	≡ Name ▲	≡ HTTP method	≡ Relative path	≡ Resource path	≡ API version	≡ Active
⚙️	🔍	Remove flag for Specific Update Set	POST	/remove_flag/{sys_id}	/api/x_661539_update_0/remove_flag_from_...	(empty)	true

4.8.2 Scripted REST Resource

Script responsible to make the REST call to the previous instance and update the flag "Ready to move to prod?" to false and the flag "Completed on Prod" to true.

Scripted REST Resource

Remove flag for Specific Update Set

#sys_wa_operation [scratchpad] [toggle label]

Update

Delete

API definition | web_service_definition

Remove Flag from Committed Update Sets

ⓘ

Application | sys_scope

Update Set Mover

ⓘ

* Name | name

Remove flag for Specific Update Set

Active | active

☒

Request routing

The route configuration specifies the 'HTTP method' and 'Relative path'. These fields determine how HTTP clients access this resource.

The relative path identifies the sub-path to this resource relative to the base API path. The relative URI can contain path parameters such as '/abc/{id}'. The requesting client specifies the id value, available to the script at runtime via the [Request API](#).

[More info](#)

* HTTP method | http_method

POST | POST

Relative path | relative_path

/remove_flag/{sys_id}

Resource path | operation_uri

/api/x_661539_update_0/remove_flag_from_committed_update_sets/remove_flag/{sys_id}

Implement the resource

Access request details including URI path parameters, query parameters, headers, and the request body using the: [Request API](#).

Configure the response including setting the HTTP status code, response body, and any response headers using the: [Response API](#).

[More info](#)

* Script | operation_script

ⓘ

1- (function process(/*RESTAPIRequest*/ request, /*RESTAPIResponse*/ response) {

2- var updateSysID = '' + request.pathParams.sys_id;

3-

4- if (updateSysID == '') {

5- response.setStatus(406);

6- response.setBody({"status": "Not Acceptable - Parameter missing"});

7- } else {

8- try {

9- var updateSetMoverUtils = new x_661539_update_0.updateSetMoverUtils();

10- var setFlagFalse = updateSetMoverUtils.setReadyForProdFalse(updateSysID);

11- response.setStatus(200);

12- response.setBody({"status": "Success"});

13-

14- } catch (e) {

15- gs.debug("REST Remove Flag from Committed Update Sets Error: " + e);

16- response.setStatus(500);

17- response.setBody({"status": "Problem on the server side"});

18- }

19-

20- }

21-

22- })(request, response);

Protection policy | sys_policy

-- None --

Security

Content Negotiation

Documentation

Select Information Classification | Uncontrolled if printed.

© Fujitsu 2022 | 7935. All rights reserved. Fujitsu and Fujitsu logo are trademarks of Fujitsu Limited registered in many jurisdictions worldwide. Other product, service and company names mentioned herein may be trademarks of Fujitsu or other companies. This document is current as of the initial date of publication and subject to be changed by Fujitsu without notice. This material is provided for information purposes only and Fujitsu assumes no liability related to its use.